



รายงานโครงงานรอบที่ 2

การศึกษา Algorithm สำหรับการทดสอบซอฟต์แวร์ และการออกแบบ Prompt สำหรับ AI

สมาชิก

นายณัฐพงศ์ กรณกิจ	673380038-9
นายธนกร ทองศรี	673380040-2
นายธนันชัย พันธราช	673380042-8
นายพีรพัฒน์ ป้องกันยา	673380053-3

เสนอ

ผศ.ดร.ชิตสุธา สุ่มเล็ก

รายงานเล่มนี้เป็นส่วนหนึ่งของรายวิชา CP353201 Software Quality Assurance
ภาคเรียนที่ 1 ปีการศึกษา 2569
สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์
มหาวิทยาลัยขอนแก่น

คำนำ

รายงานโครงงานฉบับนี้จัดทำขึ้นเพื่อศึกษาและเปรียบเทียบการสร้างกรณีทดสอบอัตโนมัติด้วยอัลกอริทึม Differential Evolution (DE) และ Ant Colony Optimization (ACO) กับการใช้เครื่องมือปัญญาประดิษฐ์ Claude และ Gemini ในการสร้างชุดทดสอบระดับหน่วยสำหรับโปรแกรมภาษา Java ด้วยชุดข้อมูล Defects4J ในรายวิชา CP353201 การประกันคุณภาพซอฟต์แวร์ โดยมีวัตถุประสงค์เพื่อประเมินความถูกต้องและความครอบคลุมของชุดทดสอบ ความสามารถในการตรวจพบข้อบกพร่อง ตลอดจนประสิทธิภาพและข้อจำกัดของแต่ละวิธี

เนื้อหาของรายงานประกอบด้วยที่มาและความสำคัญของโครงงาน วัตถุประสงค์และขอบเขตการศึกษา อัลกอริทึมและเครื่องมือปัญญาประดิษฐ์ ที่เกี่ยวข้อง ขั้นตอนการพัฒนาอัลกอริทึมและการออกแบบพารามิเตอร์ วิธีดำเนินการทดลอง การวัดและเปรียบเทียบผล รวมถึงสรุปผล ปัญหาที่พบ และข้อเสนอแนะในการพัฒนาต่อไป โดยให้ความสำคัญกับการจัดเก็บซอร์สโค้ด ชุดทดสอบ การตั้งค่า และหลักฐานการทดลองอย่างเป็นระบบ เพื่อให้สามารถตรวจสอบและทำซ้ำได้

คณะผู้จัดทำหวังว่ารายงานฉบับนี้จะเป็นประโยชน์ต่อผู้ที่สนใจด้านการประกันคุณภาพซอฟต์แวร์ การสร้างกรณีทดสอบอัตโนมัติ และการประยุกต์ใช้ปัญญาประดิษฐ์ในกระบวนการทดสอบซอฟต์แวร์ ตลอดจนเป็นแนวทางประกอบการเลือกใช้วิธีและเครื่องมือให้เหมาะสมกับลักษณะของงานที่ทำการทดสอบ หากรายงานฉบับนี้มีข้อบกพร่องประการใด คณะผู้จัดทำขอน้อมรับข้อเสนอแนะเพื่อนำไปปรับปรุงให้สมบูรณ์ยิ่งขึ้น

คณะผู้จัดทำ

นายธนกร ทองศรี

นายณนัชชัย พันธราช

นายพีรพัฒน์ กรณกิจ

นายณัฐพงศ์ ป้องกันยา

สารบัญ

คำนำ	ก
สารบัญ	ข
สารบัญรูป	ง
สารบัญตาราง	จ
บทที่ 1 บทนำ	1
1.1 ที่มาและความสำคัญของโครงงาน	1
1.2 วัตถุประสงค์ของโครงงาน	1
1.3 ขอบเขตของโครงงาน	2
1.4 ประโยชน์ที่คาดว่าจะได้รับ	2
บทที่ 2 งานที่เกี่ยวข้อง	3
2.1 Defects4J	3
2.2 Differential Evolution	3
2.3 Ant Colony Optimization	4
2.4 Claude Sonnet 5	4
2.5 Gemini 3.5 Flash Lite	5
2.6 ตัวชี้วัดสำหรับการประเมิน	5
บทที่ 3 วิธีดำเนินงาน	6
3.1 ศึกษาขอบเขตโครงงานและเตรียมชุดข้อมูล Defects4J	6
3.2 เตรียมเครื่องมือและสภาพแวดล้อมสำหรับการทดลอง	6
3.3 พัฒนาอัลกอริทึมสร้างชุดทดสอบอัตโนมัติ	7
3.3.1 การสร้างชุดทดสอบด้วย Differential Evolution	7
3.3.2 การสร้างชุดทดสอบด้วย Ant Colony Optimization	9
3.4 ออกแบบพารามิเตอร์และสร้างชุดทดสอบด้วย Claude และ Gemini	10
3.4.1 การสร้างชุดทดสอบด้วย Claude	10
3.4.2 การสร้างชุดทดสอบด้วย Gemini	11
3.5 ตรวจสอบชุดทดสอบกับโปรแกรมรุ่น Fixed และ Buggy	11
3.6 การวัดผลตามตัวชี้วัดที่กำหนด	12
3.7 รวบรวม ตรวจสอบ และเปรียบเทียบผลการทดลอง	13
บทที่ 4 ผลการทดลองและการวิเคราะห์ผล	14
4.1 ข้อมูลที่ใช้วิเคราะห์และเกณฑ์คัดเลือก	14
4.2 ความครบถ้วนของการทดลอง	14
4.3 ผลตัวชี้วัดจากรายการที่แต่ละวิธีประเมินได้	15
4.4 การเปรียบเทียบบนรายการข้อบกพร่องเดียวกัน	15
4.5 การอภิปรายผลและปัจจัยที่มีผลต่อการตีความ	16
4.5.1 ความสัมพันธ์ระหว่าง coverage กับการตรวจพบข้อบกพร่อง	16
4.5.2 ความครบถ้วนของชุดทดสอบและผลตรวจสอบ	16
4.5.3 ข้อจำกัดของการทดลอง	16

สารบัญ (ต่อ)

4.6 ข้อจำกัดของแต่ละวิธี.....	17
4.6.1 Differential Evolution.....	17
4.6.2 Ant Colony Optimization.....	17
4.6.3 Claude.....	17
4.6.4 Gemini.....	17
บทที่ 5 สรุปผลและข้อเสนอแนะ.....	18
5.1 สรุปผลการศึกษา.....	18
5.2 ข้อจำกัดของงาน.....	18
5.3 ข้อเสนอแนะสำหรับการพัฒนาต่อ.....	19
5.4 ข้อสรุป	19

สารบัญรูป

ภาพที่ 1 การสร้างชุดทดสอบด้วย Differential Evolution.....	7
ภาพที่ 2 การสร้างชุดทดสอบด้วย Ant Colony Optimization	9

สารบัญตาราง

ตารางที่ 1	นิยามตัวชี้วัดที่ใช้ในโครงการ.....	5
ตารางที่ 2	เครื่องมือที่ใช้ในกระบวนการทดลอง.....	7
ตารางที่ 3	Configuration ตั้งต้นของ DE	8
ตารางที่ 4	Configuration ตั้งต้นของ ACO.....	10
ตารางที่ 5	หลักเกณฑ์จำแนกผลการตรวจจุดทดสอบ	12
ตารางที่ 6	ตัวชี้วัดหลักที่ใช้เปรียบเทียบ	12
ตารางที่ 7	หลักฐานผลการทดลองแยกตามวิธี	13
ตารางที่ 8	สถานะผลรายข้อบกพร่องของทั้งสี่วิธี	14
ตารางที่ 9	ผลรายวิธีจากข้อมูลที่ประเมินได้ของแต่ละวิธี	15
ตารางที่ 10	ความครบถ้วนของผลในโครงการ Math จำนวน 106 ข้อบกพร่อง.....	15
ตารางที่ 11	ตัวชี้วัดของโครงการ Math จากเป้าหมายเดียวกัน 106 รายการ	16

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญของโครงการ

การทดสอบระดับหน่วยเป็นกระบวนการตรวจสอบพฤติกรรมของส่วนย่อยของซอฟต์แวร์ เช่น เมธอดหรือคลาส โดยกำหนดข้อมูลนำเข้า เงื่อนไขเริ่มต้น และผลลัพธ์ที่คาดหวังให้ชัดเจน การออกแบบกรณีทดสอบด้วยผู้พัฒนาจำเป็นต้องพิจารณาการใช้งานปกติ ค่าขอบเขต และกรณีผิดปกติ เมื่อโปรแกรมมีเงื่อนไขจำนวนมากหรือมีชนิดข้อมูลซับซ้อน การสร้างชุดทดสอบให้ครอบคลุมจึงต้องใช้เวลาและความเข้าใจโครงสร้างของโปรแกรม

แนวทางสร้างกรณีทดสอบอัตโนมัติช่วยค้นหาข้อมูลนำเข้าหรือสร้างโค้ดทดสอบเพื่อลดภาระดังกล่าว อัลกอริทึมค้นหาสามารถใช้ข้อมูลจากการรันโปรแกรมเป็นค่าประเมินคุณภาพของคำตอบ ขณะที่โมเดลภาษาขนาดใหญ่สามารถรับซอร์สโค้ดและคำอธิบายเพื่อสร้างโค้ด JUnit รวมถึงเงื่อนไขตรวจสอบผลลัพธ์ อย่างไรก็ตาม ในการสร้างไฟล์ทดสอบได้ไม่ได้รับประกันว่าไฟล์นั้นคอมไพล์ผ่าน ทดสอบได้อย่างคงที่ หรือสามารถเปิดเผยข้อบกพร่องจริงได้ จึงต้องทำการประเมินด้วยกระบวนการที่ตรวจสอบซ้ำได้

โครงการเลือก DE และ ACO เป็นตัวแทนของอัลกอริทึมสร้างกรณีทดสอบอัตโนมัติ และเลือก Claude Sonnet 5 กับ Gemini 3.5 Flash Lite เป็นเครื่องมือ AI โดยใช้ Defects4J ซึ่งเชื่อมโยงโปรแกรมรุ่นที่มีข้อบกพร่องกับรุ่นที่แก้ไขแล้วเป็นฐานในการประเมิน การเปรียบเทียบจึงพิจารณาทั้งความครอบคลุมของโค้ดและความสามารถแยกพฤติกรรมระหว่างสองรุ่น

การส่งงานรอบที่ 2 เน้นการรันจริง การวัดผล และหลักฐานที่ทำซ้ำได้ โดยเชื่อมการเลือกบั๊ก การสร้างและตรวจชุดทดสอบ กับนิยามตัวชี้วัด และแยกผลชุดทดสอบเดิมของ Defects4J ออกจากชุดทดสอบที่แต่ละวิธีสร้างขึ้น

1.2 วัตถุประสงค์ของโครงการ

1. พัฒนาและประยุกต์ใช้ DE และ ACO ในเพื่อสร้างชุดทดสอบและกรณีทดสอบระดับหน่วยสำหรับโปรแกรมภาษา Java ใน Defects4J
2. ออกแบบและนำพารอมต์ไปใช้กับ Claude และ Gemini เพื่อสร้างชุดทดสอบ JUnit ที่สอดคล้องกับคลาสเป้าหมายและข้อจำกัดของสภาพแวดล้อม
3. ตรวจสอบชุดทดสอบที่สร้างขึ้น และวัด Line coverage, Conditions coverage, Fault detection rate ตลอดจนเวลาที่ใช้ภายใต้นิยามที่ระบุชัดเจน
4. วางหลักเกณฑ์เปรียบเทียบทั้งสี่วิธี วิเคราะห์ข้อจำกัด และจัดเก็บหลักฐานการทดลองในรูปแบบที่ผู้อื่นสามารถตรวจสอบและทำซ้ำได้

1.3 ขอบเขตของโครงการ

1. ศึกษาการสร้างชุดทดสอบและกรณีทดสอบอัตโนมัติในระดับหน่วย (Unit Testing) สำหรับโปรแกรมภาษา Java โดยใช้ JUnit เป็นกรอบการทดสอบ
2. ใช้โครงการและข้อบกพร่องจากชุดข้อมูล Defects4J จาก 854 ข้อบกพร่องจาก 17 โครงการ
3. ศึกษาและพัฒนาอัลกอริทึมสร้างกรณีทดสอบอัตโนมัติของ Differential Evolution (DE) และ Ant Colony Optimization (ACO)
4. ศึกษาและออกแบบพอร์มต์สำหรับเครื่องมือปัญญาประดิษฐ์ Claude และ Gemini
5. ตรวจสอบความถูกต้องของชุดทดสอบด้วยโปรแกรมรุ่นที่แก้ไขข้อบกพร่องแล้ว (Fixed Revision) และนำชุดทดสอบที่ผ่านการตรวจสอบไปทดสอบกับรุ่นที่มีข้อบกพร่อง (Buggy Revision)
6. ประเมินชุดทดสอบด้วยตัวชี้วัด ได้แก่ Line coverage, Conditions coverage, Fault detection rate และเวลาที่ใช้ในการสร้างและประเมินชุดทดสอบ
7. จัดเก็บซอร์สโค้ด ชุดทดสอบ พอร์มต์ การตั้งค่า ผลการทดลอง และบันทึกการทำงานใน GitHub อย่างเป็นระบบ

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. เข้าใจการสร้างกรณีทดสอบอัตโนมัติด้วย DE, ACO, Claude และ Gemini
2. ได้แนวทางตรวจสอบความถูกต้องและประเมินคุณภาพของชุดทดสอบ
3. ทราบข้อดีและข้อจำกัดของแต่ละวิธีจากการทดลอง
4. สามารถเลือกใช้วิธีสร้างชุดทดสอบให้เหมาะสมกับงาน
5. ได้ข้อมูลและหลักฐานการทดลองที่ตรวจสอบและทำซ้ำได้

บทที่ 2

งานที่เกี่ยวข้อง

2.1 Defects4J

Defects4J เป็นฐานข้อมูลข้อบกพร่องจริงจากโครงการซอฟต์แวร์ภาษา Java พร้อมเครื่องมือสนับสนุนการทดลองด้านการทดสอบซอฟต์แวร์ โดยข้อบกพร่องแต่ละรายการเชื่อมโยงกับโปรแกรมสองรุ่น ได้แก่ รุ่นที่มีข้อบกพร่อง (Buggy Revision) และรุ่นที่แก้ไขข้อบกพร่องแล้ว (Fixed Revision) ทำให้สามารถนำชุดทดสอบเดียวกันไปตรวจสอบและเปรียบเทียบพฤติกรรมของโปรแกรมทั้งสองรุ่นได้

เครื่องมือของ Defects4J รองรับการดึงซอร์สโค้ดของรุ่นที่ต้องการ การคอมไพล์ การรันชุดทดสอบ และการวัดความครอบคลุมของโค้ด นอกจากนี้ยังมีข้อมูลประกอบ เช่น คลาสที่ถูกแก้ไขและชุดทดสอบที่เกี่ยวข้องกับข้อบกพร่อง ช่วยให้ผู้ใช้ศึกษาหาเป้าหมายและจัดเตรียมการทดลองได้อย่างเป็นระบบ

โครงการนี้ใช้ Defects4J เป็นฐานสำหรับประเมินชุดทดสอบที่สร้างด้วย DE, ACO, Claude และ Gemini โดยตรวจสอบชุดทดสอบกับรุ่น Fixed ก่อน แล้วนำชุดที่ผ่านไปทดสอบกับรุ่น Buggy หากกรณีทดสอบผ่านบนรุ่น Fixed แต่ไม่ผ่านบนรุ่น Buggy และความล้มเหลวไม่ได้เกิดจากปัญหาสภาพแวดล้อม จะถือเป็นหลักฐานว่ากรณีทดสอบนั้นสามารถเปิดเผยข้อบกพร่องได้ พร้อมทั้งวัดความครอบคลุมของโค้ดเพื่อประกอบการเปรียบเทียบแต่ละวิธี

2.2 Differential Evolution

Differential Evolution (DE) เป็นอัลกอริทึมค้นหาคำตอบที่เหมาะสมโดยใช้ประชากรของคำตอบ แต่ละคำตอบแทนด้วยเวกเตอร์ตัวเลข และประเมินคุณภาพด้วยค่าความเหมาะสม (Fitness) จากนั้นปรับปรุงคำตอบผ่านกระบวนการกลายพันธุ์ การผสมข้าม และการคัดเลือก เพื่อค้นหาคำตอบที่ดีขึ้นจนถึงเงื่อนไขสิ้นสุด

โครงการนี้ใช้ DE รูปแบบ DE/rand/1/bin ซึ่งมีขั้นตอนหลักดังนี้

1. สร้างประชากรเริ่มต้น กำหนดชุดคำตอบภายในขอบเขตข้อมูลที่กำหนด
2. กลายพันธุ์ (Mutation) สุ่มคำตอบสามตัวที่แตกต่างกัน แล้วนำผลต่างของคำตอบสองตัวคูณด้วยค่าความสุ่ม และบวกกับคำตอบตัวที่เหลือ
3. ผสมข้าม (Crossover) ผสมองค์ประกอบของคำตอบใหม่กับคำตอบเดิมตามความน่าจะเป็น
4. คัดเลือก (Selection) เปรียบเทียบค่า Fitness และเก็บคำตอบที่มีคุณภาพดีกว่าหรือเท่ากันไว้ในประชากร
5. ทำซ้ำ ดำเนินการจนใช้งบประมาณการประเมินครบหรือถึงเงื่อนไขหยุดที่กำหนด

ในการสร้างกรณีทดสอบ DE ใช้ค้นหาข้อมูลนำเข้าสำหรับเมธอดเป้าหมาย โดยแปลงเวกเตอร์คำตอบเป็นข้อมูลที่โปรแกรมรับได้ และใช้ความครอบคลุมของโค้ดเป็นส่วนหนึ่งของค่า Fitness ข้อมูลที่ค้นหาได้จะถูกนำไปสร้างชุดทดสอบ JUnit และตรวจสอบกับโปรแกรมรุ่น Fixed และ Buggy เพื่อประเมินความสามารถในการเปิดเผยข้อบกพร่อง

2.3 Ant Colony Optimization

Ant Colony Optimization (ACO) เป็นอัลกอริทึมค้นหาคำตอบที่ได้รับแรงบันดาลใจจากพฤติกรรมกรหาอาหารของมด โดยมดจะทิ้งสารฟีโรโมนไว้ตามเส้นทางเพื่อเป็นข้อมูลให้มดตัวอื่นเลือกเดิน อัลกอริทึมนี้มาใช้ร่วมกับข้อมูลฮิสตริก ซึ่งช่วยประเมินความน่าสนใจของทางเลือก เพื่อค้นหาคำตอบที่เหมาะสมผ่านการทำงานซ้ำหลายรอบ

การทำงานของ ACO ประกอบด้วยขั้นตอนหลักดังนี้

1. กำหนดค่าเริ่มต้น เตรียมกราฟ จำนวนมด จำนวนรอบ และค่าฟีโรโมนเริ่มต้นบนเส้นเชื่อม
2. สร้างเส้นทาง ให้มดแต่ละตัวเลือกเส้นทางโดยพิจารณาฟีโรโมนและข้อมูลฮิสตริก
3. ประเมินคำตอบ ตรวจสอบคุณภาพของเส้นทางหรือคำตอบที่มดสร้างขึ้นตามเกณฑ์ที่กำหนด
4. ปรับปรุงฟีโรโมน เพิ่มฟีโรโมนให้กับเส้นทางตามคุณภาพของคำตอบ และลดฟีโรโมนด้วยกระบวนการระเหย

5. ทำซ้ำ ดำเนินการจนถึงจำนวนรอบหรือบรรลุเป้าหมายที่กำหนด

ในการสร้างกรณีทดสอบ โครงการประยุกต์ใช้ ACO กับกราฟควบคุมการทำงาน (Control Flow Graph) โดยให้มดค้นหาเส้นทางไปยังเงื่อนไขเป้าหมาย แล้วสังเคราะห์ข้อมูลนำเข้าให้สอดคล้องกับเส้นทางนั้น ข้อมูลที่ผ่านการตรวจสอบจะถูกนำไปสร้างชุดทดสอบ JUnit และทดสอบกับโปรแกรมรุ่น Fixed และ Buggy เพื่อประเมินความครอบคลุมของโค้ดและความสามารถในการเปิดเผยข้อบกพร่อง

2.4 Claude Sonnet 5

Claude เป็นโมเดลภาษาขนาดใหญ่ที่พัฒนาโดย Anthropic สามารถประมวลผลข้อความและซอร์สโค้ดเพื่อตอบคำถาม อธิบายการทำงานของโปรแกรม และสร้างโค้ดตามคำสั่ง ในโครงการนี้นำ Claude มาใช้สร้างกรณีทดสอบระดับหน่วย โดยให้ซอร์สโค้ดของคลาสเป้าหมายพร้อมพอร์มต์ที่กำหนดข้อจำกัดและรูปแบบชุดทดสอบที่ต้องการ

กระบวนการสร้างชุดทดสอบด้วย Claude ในโครงการมีขั้นตอนหลักดังนี้

1. เตรียมข้อมูลเป้าหมาย รวบรวมซอร์สโค้ด ชื่อคลาส และชื่อแพ็คเกจของโปรแกรมที่ต้องการทดสอบ
2. ประกอบพอร์มต์ ระบุให้สร้างชุดทดสอบ JUnit 4 พร้อมกฎด้านความเข้ากันได้ การเลือกค่าขอบเขต และการกำหนดผลลัพธ์ที่คาดหวัง
3. สร้างโค้ดทดสอบ ส่งพอร์มต์และซอร์สโค้ดให้ Claude แล้วจัดเก็บคำตอบและข้อมูลการเรียกใช้งาน
4. ตรวจสอบชุดทดสอบ ตรวจสอบรูปแบบโค้ด คอมไพล์ และรันทดสอบกับโปรแกรมรุ่น Fixed เพื่อคัดกรองกรณีทดสอบที่ไม่ถูกต้อง
5. ประเมินผล นำชุดทดสอบที่ผ่านการตรวจไปทดสอบกับรุ่น Buggy และวัดความครอบคลุมของโค้ดกับความสามารถในการเปิดเผยข้อบกพร่อง

การออกแบบพอร์มต์ให้มีความสำคัญกับการกำหนดผลลัพธ์ที่คาดหวังจากเอกสารหรือสัญญาของเมธอด เพื่อช่วยลดการสร้างกรณีทดสอบที่ยืนยันพฤติกรรมผิดของโปรแกรม นอกจากนี้ยังกำหนดให้พิจารณาข้อมูลขอบเขตและกรณีผิดปกติ รวมถึงหลีกเลี่ยงการพึ่งพาเวลาปัจจุบันหรือสภาพแวดล้อมภายนอกที่ทำให้ผลทดสอบเปลี่ยนแปลงได้

2.5 Gemini 3.5 Flash Lite

Gemini เป็นโมเดลภาษาขนาดใหญ่ที่พัฒนาโดย Google สามารถประมวลผลข้อความและซอร์สโค้ดเพื่อตอบคำถาม อธิบายการทำงานของโปรแกรม และสร้างโค้ดตามคำสั่ง ในโครงงานนี้นำ Gemini มาใช้สร้างกรณีทดสอบระดับหน่วย โดยให้ซอร์สโค้ดของคลาสเป้าหมายพร้อมพร้อมต์ที่กำหนดข้อจำกัดและรูปแบบชุดทดสอบที่ต้องการ

กระบวนการสร้างชุดทดสอบด้วย Gemini ในโครงงานมีขั้นตอนหลักดังนี้

1. เตรียมข้อมูลเป้าหมาย รวบรวมซอร์สโค้ด ชื่อคลาส และชื่อแพ็คเกจของโปรแกรมที่ต้องการทดสอบ
2. ประกอบพร้อมต์ ระบุให้สร้างชุดทดสอบ JUnit 4 พร้อมกฎด้านความเข้ากันได้ การเลือกค่าขอบเขต และการกำหนดผลลัพธ์ที่คาดหวัง
3. สร้างโค้ดทดสอบ ส่งพร้อมต์และซอร์สโค้ดให้ Gemini ผ่านบริการ KKU IntelSphere แล้วจัดเก็บโค้ดทดสอบที่ได้รับ
4. ตรวจสอบชุดทดสอบ ตรวจสอบรูปแบบโค้ด คอมไพล์ และรันทดสอบกับโปรแกรมรุ่น Fixed เพื่อคัดกรองกรณีทดสอบที่ไม่ถูกต้อง
5. ประเมินผล นำชุดทดสอบที่ผ่านการตรวจไปทดสอบกับรุ่น Buggy และวัดความครอบคลุมของโค้ดกับความสามารถในการเปิดเผยข้อบกพร่อง

การออกแบบพร้อมต์ให้ความสำคัญกับการเลือกข้อมูลขอบเขตและกรณีผิดปกติของชุดทดสอบ เช่น ค่าศูนย์ ค่าลบ ค่าสูงสุดต่ำสุด สตริงว่าง ค่า null และคอลเลกชันว่าง พร้อมกำหนดให้ใช้ API ที่มีอยู่จริงและตรวจสอบผลลัพธ์อย่างเหมาะสม นอกจากนี้ยังระบุข้อจำกัดด้านไวยากรณ์ Java และรูปแบบ JUnit เพื่อให้โค้ดทดสอบสอดคล้องกับสภาพแวดล้อมของโครงการ

2.6 ตัวชี้วัดสำหรับการประเมิน

โครงงานนี้ใช้ตัวชี้วัด 3 ด้านในการประเมินชุดทดสอบที่สร้างด้วย DE, ACO, Claude และ Gemini ได้แก่ Line Coverage, Condition Coverage และ Fault Detection Rate โดยวัดความครอบคลุมจากชุดทดสอบที่ผ่านการตรวจสอบแล้ว และพิจารณาการตรวจพบข้อบกพร่องจากผลทดสอบของโปรแกรมรุ่น Fixed และ Buggy

ตารางที่ 1 นิยามตัวชี้วัดที่ใช้ในโครงงาน

ตัวชี้วัด	นิยามและเงื่อนไข
Line coverage	อัตราส่วนของบรรทัดคำสั่งที่ถูกเรียกทำงานระหว่างการทำทดสอบ ต่อจำนวนบรรทัดคำสั่งที่วัดได้ทั้งหมด
condition coverage	อัตราส่วนของผลลัพธ์จริงและเท็จของเงื่อนไขย่อยที่ถูกทดสอบ ต่อจำนวนผลลัพธ์จริงและเท็จที่เป็นไปได้ทั้งหมดของเงื่อนไขย่อย
Fault detection rate	อัตราส่วนของจำนวนข้อบกพร่องที่ชุดทดสอบตรวจพบ ต่อจำนวนข้อบกพร่องที่ประเมินผลได้ทั้งหมด

บทที่ 3

วิธีดำเนินงาน

ในการทำโครงงานครั้งนี้ ผู้ศึกษาได้ดำเนินการตามขั้นตอนดังนี้

1. ศึกษาขอบเขตโครงงานและเตรียมชุดข้อมูล Defects4J
2. เตรียมเครื่องมือและสภาพแวดล้อมสำหรับการทดลอง
3. พัฒนา DE และ ACO เพื่อสร้างชุดทดสอบอัตโนมัติ
4. ออกแบบพารามิเตอร์และสร้างชุดทดสอบด้วย Claude และ Gemini
5. ตรวจสอบชุดทดสอบกับโปรแกรมรุ่น Fixed และ Buggy
6. วัดผลตามตัวชี้วัดที่กำหนด
7. รวบรวม ตรวจสอบ และเปรียบเทียบผลการทดลอง

3.1 ศึกษาขอบเขตโครงงานและเตรียมชุดข้อมูล Defects4J

ศึกษาวิธีสร้างชุดทดสอบและกรณีทดสอบระดับหน่วยด้วยอัลกอริทึม 2 วิธีและเครื่องมือ AI 2 วิธี จากนั้นนำผลไปทดลองกับ Java projects ใน Defects4J และเปรียบเทียบประสิทธิภาพ Defects4J จัดเก็บโปรแกรมรุ่นที่มีข้อบกพร่อง (Buggy Revision) ควบคู่กับรุ่นที่แก้ไขแล้ว (Fixed Revision) จึงสามารถใช้ชุดทดสอบเดียวกันตรวจสอบความแตกต่างของพฤติกรรมได้

กำหนดหน่วยทดลองเป็นวิธีสร้างชุดทดสอบหนึ่งวิธีต่อข้อบกพร่องหนึ่งรายการต่อรอบการรันหนึ่งรอบ โดยระบุชื่อโครงการ รหัสข้อบกพร่อง รุ่นโปรแกรม และหมายเลขรอบการรันให้ชัดเจน ชุดเป้าหมายของโครงงานประกอบด้วย 854 ข้อบกพร่องจาก 17 โครงการ ได้แก่ Chart, Cli, Closure, Codec, Collections, Compress, Csv, Gson, JacksonCore, JacksonDatabind, JacksonXml, Jsoup, JXPath, Lang, Math, Mockito และ Time

สำหรับแต่ละรายการ ใช้คำสั่งของ Defects4J เตรียมโปรแกรมทั้งสองรุ่น แล้วดึงข้อมูลคลาสที่มีการแก้ไข (modified classes) และ classpath ของการทดสอบมาเป็นข้อมูลให้ตัวสร้างชุดทดสอบ การระบุคลาสเป้าหมายจากข้อมูลของบักช่วยกำหนดขอบเขตการค้นหาและการวัดผลให้สัมพันธ์กับข้อบกพร่องจริง ไม่ใช่ความครอบคลุมทั้งโครงการแทนความครอบคลุมของคลาสที่แก้ไข

3.2 เตรียมเครื่องมือและสภาพแวดล้อมสำหรับการทดลอง

การรัน pipeline ใช้ Linux หรือ Windows Subsystem for Linux (WSL) ที่ติดตั้ง Defects4J, JDK 11, Python และเครื่องมือที่ Defects4J ต้องใช้ โดยตรวจสอบว่าคำสั่ง checkout, compile, test และ coverage ทำงานได้ก่อนเริ่มชุดทดลองขนาดใหญ่

ACO ใช้ AcoSearch.java, AcoOracle.java และ AcoReplay.java ร่วมกับ JaCoCo เพื่อรับข้อมูลการเข้าถึงโค้ดระหว่างค้นหา ก่อนใช้งานต้องเตรียมไลบรารีและคอมไพล์ตัวสร้างชุดทดสอบตาม Code/setup_lib.sh

และ Code/build.sh ส่วนการวัดผลท้ายกระบวนการใช้คำสั่ง coverage ของ Defects4J ซึ่งอ่านผล Cobertura จาก modified classes

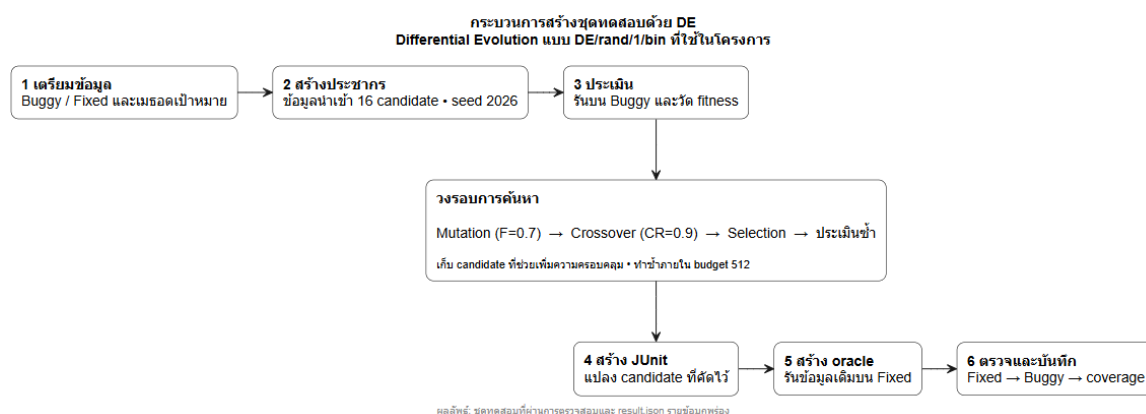
DE ใช้ซอร์สและตัวควบคุมในโฟลเดอร์ Differential Evolution (DE) โดยกำหนดค่าการค้นหาจากไฟล์ defaults.json ส่วน Claude และ Gemini ใช้พร้อมดักกับ pipeline ของแต่ละวิธี การรันโมเดลต้องบันทึก backend ชื่อโมเดลที่ร้องขอ รุ่นที่ตอบกลับเมื่อมี และการตั้งค่าที่มีผลต่อผลลัพธ์ ข้อมูลสำหรับเรียกบริการเก็บในตัวแปรสภาพแวดล้อม ไม่รวมไว้ในผลทดลองที่เผยแพร่

ตารางที่ 2 เครื่องมือที่ใช้ในกระบวนการทดลอง

เครื่องมือ	หน้าที่
Defects4J	เตรียมคู่รุ่น Buggy และ Fixed คอมไพล์ รันทดสอบ และวัด coverage
JUnit	เป็นรูปแบบชุดทดสอบระดับหน่วยที่สร้างและรันกับโปรแกรม Java
JaCoCo	ให้ข้อมูล probe ระหว่างการค้นหาของ ACO บนรุ่น Buggy
Cobertura ผ่าน Defects4J	วัด Line Coverage และ Condition Coverage หลังตรวจชุดทดสอบ
Python และ Java	ควบคุม pipeline ค้นหา สร้างชุดทดสอบ และบันทึกผล
Claude และ Gemini	สร้างโค้ด JUnit จากซอร์สโค้ดและพร้อมดักของแต่ละวิธี

3.3 พัฒนาอัลกอริทึมสร้างชุดทดสอบอัตโนมัติ

3.3.1 การสร้างชุดทดสอบด้วย Differential Evolution



ภาพที่ 1 การสร้างชุดทดสอบด้วย Differential Evolution

DE ที่ใช้ในโครงการเป็นรูปแบบ DE/rand/1/bin เริ่มจากสร้างประชากรของเวกเตอร์ข้อมูลนำเข้า แล้วปรับเวกเตอร์ด้วยการกลายพันธุ์ การผสมข้าม และการคัดเลือก ค่าตอบที่สร้างขึ้นถูกแปลงเป็นข้อมูลสำหรับเรียกเมธอดเป้าหมายและประเมินจากความครอบคลุมของโค้ดที่รันได้จริง ในปัจจุบันค่า fitness ผสมความครอบคลุมของแขนงที่เครื่องมือค้นหาใช้กับความครอบคลุมของบรรทัด ทั้งนี้ค่า fitness ระหว่างค้นหาเป็นเกณฑ์ภายใน ไม่ใช่ตัวชี้วัดผลลัพธ์ทั้งสามที่ใช้สรุปเปรียบเทียบในรายงาน

ตัวควบคุม DE เตรียมข้อมูลของคลาสและเมธอดเป้าหมาย สร้างกรณีทดสอบจากข้อมูลที่ค้นพบ และตรวจผลกับโปรแกรมสองรุ่น การกำหนดค่าประชากร งบประมาณ จำนวนกรณีทดสอบ และ seed ต้องเก็บร่วมกับรหัสการทดลอง ผล snapshot ที่มีอยู่ในคลังโครงการมาจาก algorithm version มากกว่าหนึ่งรุ่น จึงต้องตรวจ version รายการก่อนนำมาเปรียบเทียบ

ขั้นแรกของ DE คือเตรียม checkout ของรุ่น Buggy และ Fixed แล้วอ่านข้อมูล modified classes เพื่อเลือกเมธอดเป้าหมาย ตัวค้นหาสร้างเวกเตอร์จำนวนจริงภายในขอบเขตที่กำหนด ก่อนปัดเป็นจำนวนเต็มและนำไปสร้างข้อมูลสำหรับเรียกเมธอด การประเมินคำตอบใช้ JaCoCo ระหว่างค้นหาเพื่อดูการเข้าถึงบรรทัดและแขนง พร้อมเก็บข้อมูลของ candidate ที่เคยประเมินเพื่อไม่รันซ้ำโดยไม่จำเป็น

เมื่อตัวค้นหาพบ candidate ที่เรียก จะคัดเลือกชุดที่เพิ่มความครอบคลุมและความหลากหลายของข้อมูล แล้ว replay บนรุ่น Fixed เพื่อสร้างผลที่คาดหวังสำหรับ assertion หากผลบน Fixed ไม่คงที่หรือไม่สามารถรันซ้ำได้ จะไม่ใช่ candidate นั้นเป็น oracle กรณีทดสอบที่ได้ถูกเขียนเป็นชุด JUnit ร่วมกับส่วนช่วย replay และเก็บทั้งชุดที่สร้างกับข้อมูล candidate เพื่อให้ตรวจย้อนกลับได้

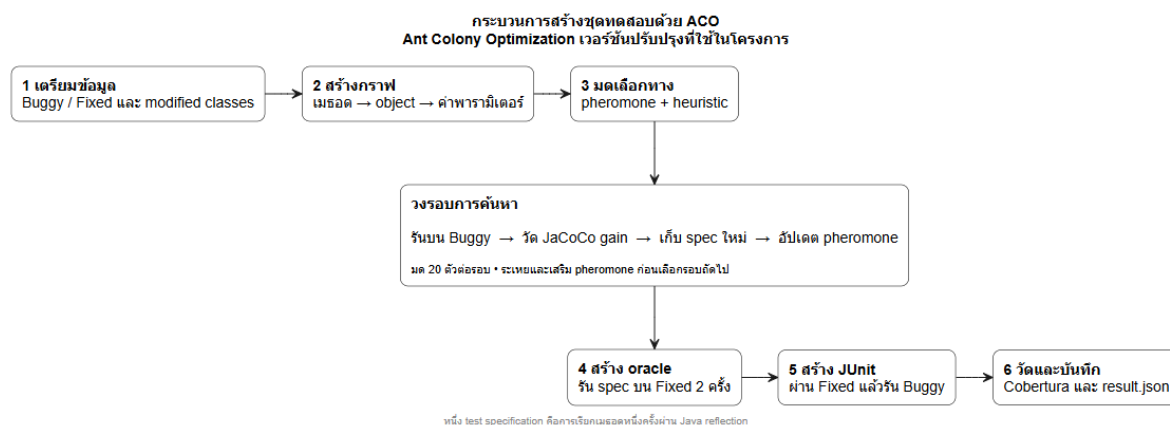
pipeline ของ DE ตรวจชุดทดสอบบนรุ่น Fixed โดยรันซ้ำและตัดกรณีที่ไม่น่าสนใจออก จากนั้นใช้ชุดที่เหลือรันบนรุ่น Buggy ซ้ำเพื่อยืนยันกรณีที่เปิดเผยความต่าง ขั้นสุดท้ายวัด coverage ของชุดที่ผ่านการตรวจด้วย Defects4J หาก Cobertura ไม่สามารถรายงานได้ pipeline มีขั้นตอนใช้ JaCoCo เป็นทางเลือก ซึ่งต้องระบุชนิดเครื่องมือไว้ในผลก่อนนำไปเปรียบเทียบกับ Condition Coverage จาก Cobertura

ตารางที่ 3 Configuration ตั้งต้นของ DE

พารามิเตอร์	ค่า	หน้าที่
population / budget	16 / 512	ขนาดประชากรและงบประมาณประเมิน
mutation / crossover	0.7 / 0.9	ค่าควบคุมการกลายพันธุ์และการผสมข้าม
input_range / seed	1000 / 2026	ขอบเขตเวกเตอร์และค่าเริ่มต้นสุ่ม
runs / workers	1 / 4	จำนวนรอบและงานที่ทำพร้อมกัน
max_tests_per_method	16	จำนวนกรณีทดสอบสูงสุดต่อเมธอด
method_timeout / job_timeout	300 / 1500 วินาที	ขีดจำกัดเวลาระดับเมธอดและงาน

การใช้ค่าตั้งต้นในตารางที่ 3 ต้องผูกกับผลของการรันนั้นจริง เพราะ repository มีผลจากหลายเวอร์ชันของ DE ไฟล์ configuration.json, environment.json และ manifest.json ที่บันทึกกับ experiment จึงเป็นหลักฐานหลักสำหรับการทำซ้ำและการระบุขอบเขตผล

3.3.2 การสร้างชุดทดสอบด้วย Ant Colony Optimization



ภาพที่ 2 การสร้างชุดทดสอบด้วย Ant Colony Optimization

ACO ทำการสร้างชุดทดสอบแบบทั่วไปสำหรับคลาสที่มีการแก้ไขใน Defects4J ใช้ Java reflection สืบหาเมธอดและ constructor ที่เรียกได้ แทนการกำหนดกราฟสำหรับ NumberUtils ของ Lang-1 โดยเฉพาะกราฟการตัดสินใจของกรณีทดสอบหนึ่งตัวเริ่มจากเลือกเมธอดหรือ constructor หากเป็น instance method จะเลือกวิธีสร้างวัตถุผู้รับคำสั่ง จากนั้นเลือกข้อมูลของพารามิเตอร์ทีละตัว แล้วสร้างวัตถุพารามิเตอร์ซ้อนลงไปภายใต้ขีดจำกัดความลึกที่กำหนด

ตัวเลือกข้อมูลนำเข้าประกอบด้วยค่าปกติและค่าขอบ เช่น 0, -1, ค่าสูงสุดและต่ำสุด, NaN, สตริงว่าง, null และอาเรย์ว่าง รวมถึงค่าคงที่จาก class file และวัตถุสำเร็จรูปบางชนิดใน JDK มดแต่ละตัวเลือกเส้นทางตามฟีโรโมนและค่า heuristic โดยเส้นของเมธอดที่ยังไม่ถูกลองจะได้รับน้ำหนักเพิ่ม ส่วนเมธอดที่ค้นหาแล้วไม่พบ coverage ใหม่หลายครั้งจะลดความน่าสนใจ และหยุดเลือกเมื่อพบ timeout ตามเกณฑ์ในซอร์ส

ระหว่างการค้นหา AcoSearch ทำงานบน Buggy Revision โดยใช้ probe ของ JaCoCo เป็นข้อมูลย้อนกลับ หากกรณีทดสอบเข้าถึง probe ใหม่ หรือให้ผลลัพธ์ใหม่ของเมธอด จะเก็บข้อกำหนดการเรียกใช้งานไว้ใน archive หลังมดแต่ละรอบทำงานเสร็จ ระบบลดฟีโรโมนบนเส้นทางทั้งหมด แล้วเพิ่มฟีโรโมนให้เส้นทางที่สร้างผลลัพธ์มีประโยชน์ กระบวนการสิ้นสุดเมื่อครบจำนวนรอบที่คำนวณตามจำนวนเมธอดหรือถึงรอบเวลาที่กำหนด

หลังค้นหาจะนำ test specification ไปทำงานบน Fixed Revision สองครั้งด้วย AcoOracle เพื่อเก็บผลที่คงที่เป็นค่าที่คาดหวัง AcoReplay แปลงผลการเรียกเมธอดเป็นข้อความที่ประกอบด้วยค่าที่คืนกลับ สถานะของวัตถุ หรือชนิดข้อยกเว้นตามกรณี จากนั้นสร้าง acogen/AcoGeneratedTest.java ในรูปแบบ JUnit โดยเปรียบเทียบผลที่บันทึกจาก Fixed Revision กับผลจากการรัน specification เดียวกัน

ตารางที่ 4 Configuration ตั้งต้นของ ACO

พารามิเตอร์	ค่า	ความหมาย
number_of_ants	20	จำนวนมดในหนึ่งรอบ
iterations	150	จำนวนรอบตั้งต้น โดยตัวค้นหาเพิ่มตามจำนวนเมธอดได้
alpha / beta	1.0 / 2.0	น้ำหนักฟีโรโมนและ heuristic
evaporation_rate	0.25	อัตราการระเหยของฟีโรโมน
initial_pheromone	1.0	ค่าฟีโรโมนเริ่มต้น
random_seed	1	ค่าเริ่มต้น รอบที่ N ใช้ seed เพิ่มตามหมายเลขรอบ
time_budget_seconds	30	กรอบเวลาสำหรับการค้นหาต่อบั๊ก
call_timeout_ms	1500	ขีดจำกัดต่อการเรียกหนึ่งครั้ง
max_tests	150	ค่าควบคุมการเก็บ archive; ซอร์สอาจเก็บกรณี que เพิ่ม coverage เกินค่านี้ได้
max_object_depth	2	ความลึกสูงสุดในการสร้างวัตถุพารามิเตอร์
extra_tests_per_method	5	กรณีผลลัพธ์ใหม่ที่เก็บเพิ่มต่อเมธอด
evaluations_per_method	12	จำนวนการประเมินเป้าหมายต่อเมธอด

ค่าตารางที่ 4 มาจาก Configuration/aco_config.properties การรันจริงต้องตรวจค่าใน result.json ของแต่ละบั๊กด้วย โดยเฉพาะ seed และจำนวนรอบที่ดำเนินการ ทั้งนี้ max_tests เป็นค่าควบคุม archive ภายในตัวค้นหา มิใช่เพดานจำนวนไฟล์กรณีทดสอบหลังคัดกรองเสมอไป

3.4 ออกแบบพรมตและสร้างชุดทดสอบด้วย Claude และ Gemini

โดยกำหนดให้ซอร์สโค้ดของคลาสเป้าหมาย ชื่อแพ็คเกจ ชื่อคลาส และข้อจำกัดของการรันเป็นข้อมูลในพรมตสำหรับโมเดลทั้งสองวิธี โดยกำหนดผลลัพธ์เป็นโค้ด Java สำหรับชุดทดสอบด้วย JUnit และให้พิจารณาข้อมูลขอบเขต กรณีผิดปกติ และผลที่คาดหวังตามพฤติกรรมของเมธอด เพื่อให้ชุดทดสอบมี assertion ที่ใช้ตรวจสอบความผิดพลาดได้

3.4.1 การสร้างชุดทดสอบด้วย Claude

Claude ใช้ system prompt ในการกำหนดกฎด้านการคอมไพล์ ความคงที่ของผลทดสอบ การออกแบบ oracle และรูปแบบคำตอบ จากนั้นนำซอร์สโค้ดมาประกอบกับ user template pipeline บันทึกพรมตที่ใช้จริง คำตอบดิบ โค้ดหลังปรับรูปแบบ และ metadata ของการสร้าง เช่น รุ่นโมเดล backend และเวลาที่ใช้ ก่อนส่งให้ตัวตรวจสอบของโครงการ

ขั้นเตรียมข้อมูลของ Claude อ่านซอร์สของคลาสเป้าหมายจาก dataset แล้วดึงชื่อ package และชื่อคลาสมาประกอบกับ Prompt/system_prompt.txt และ Prompt/template_prompt.txt ภายในพารามิเตอร์บุให้ใช้ JUnit 4 และไวยากรณ์ที่เข้ากันได้กับโครงการเป้าหมาย หลักเลี้ยง API ที่ไม่มีอยู่และข้อมูลที่ทำให้ผลทดสอบเปลี่ยนตามเวลา รวมทั้งให้สร้างค่าคาดหวังจาก Javadoc หรือสัญญาของเมธอดแทนการยึดพฤติกรรมผิดของรุ่น Buggy

ตัวสร้าง Claude เรียกโมเดลตาม backend ที่เลือกใน configuration ซึ่งรองรับ KKU IntelSphere, CLI และ API โดย backend ที่ใช้จริงของแต่ละบักอาจต่างจากค่าเริ่มต้น หลังได้รับคำตอบ pipeline ปรับรูปแบบโค้ด เช่น ตัดเครื่องหมาย code fence แก่รูปแบบ JUnit หรือชื่อคลาสที่ไม่สอดคล้องกับไฟล์ และบันทึกทั้งคำตอบดิบกับไฟล์หลังปรับเพื่อแยกข้อผิดพลาดจากโมเดลออกจากผลของขั้นตอนแก้รูปแบบ

validator ของ Claude เตรียมโปรแกรมรุ่น Fixed แล้วรันชุดทดสอบเพื่อตัด test method ที่ไม่ผ่านออก ต่อมารันชุดที่ผ่านการตรวจบนรุ่น Buggy เพื่อดูความล้มเหลวที่เปิดเผยขึ้น ก่อนวัด Line Coverage และ Condition Coverage ของ modified classes หลักฐานรายการประกอบด้วย generation.json, ชุดทดสอบที่สร้าง, validated_tests, result.json และบันทึกของการทดสอบทั้งสองรุ่น

3.4.2 การสร้างชุดทดสอบด้วย Gemini

Gemini ใช้ template prompt ที่ระบุข้อจำกัด JUnit 4 การใช้ API ที่มีอยู่จริง และกรณีขอบเขต เช่น null คำว่าง ค่าต่ำสุดหรือสูงสุด และโครงสร้างข้อมูลว่าง pipeline เรียกบริการ KKU IntelSphere เพื่อสร้างโค้ดทดสอบ จากนั้นบันทึกโค้ดและส่งให้ validator ตรวจบน Defects4J ความถูกต้องของโค้ดและ assertion ตัดสินจากการคอมไพล์และผลการรัน มิใช่จากคำสั่งในพารามิเตอร์เพียงอย่างเดียว

ขั้นเตรียมข้อมูลของ Gemini ใช้ซอร์ส Java ของคลาสเป้าหมายภายใต้ dataset/<Project>_<BugID>/ แล้วประกอบกับ Prompt/template_prompt.txt เพื่อส่งให้ pipeline_gemini.py พร้อมตัวเลือกกรณีปกติและกรณีขอบเขต เช่น null สตริงว่าง ค่า overflow และคอลเล็กชันว่าง พร้อมกำหนดผลลัพธ์เป็นไฟล์ Java ที่พร้อมนำไปตรวจสอบ

ตัวสร้างส่งคำขอไปยัง KKU IntelSphere ตาม endpoint และชื่อโมเดลที่ระบุใน pipeline แล้วบันทึกชุดทดสอบไว้ในโฟลเดอร์ test/ ตัวตรวจสอบ validate.py ใช้ Defects4J รันทดสอบบนรุ่น Buggy และ Fixed เก็บผลรายบักใน result.json และบันทึกค่า coverage กับสถานะการตรวจพบข้อบกพร่อง เมื่อวิเคราะห์ผลต้องอ่าน status_detail และ fault_detected ร่วมกัน เพราะรายการที่ยังสรุปไม่ได้ไม่ควรถูกนับเป็นการทดสอบที่ไม่พบข้อบกพร่อง

3.5 ตรวจสอบชุดทดสอบกับโปรแกรมรุ่น Fixed และ Buggy

เมื่อได้ชุดทดสอบจากแต่ละวิธี จะตรวจว่ามีไฟล์ Java ที่สร้างได้จริงและสามารถคอมไพล์ร่วมกับโปรแกรมเป้าหมาย ก่อนใช้ผลในการเปรียบเทียบ การตรวจขั้นแรกใช้ Fixed Revision เพื่อตัดกรณีทดสอบที่ให้ผลผิดหรือทำงานไม่เสถียรออก ใน pipeline ACO จะรันชุดที่สร้างบน Fixed Revision แล้วตัด test method ที่ไม่ผ่านออก โดยเก็บชุดหลังคัดกรองไว้ใน validated_tests และบันทึกรายชื่อกรณีที่ไม่ผ่านไว้ใน logs

นำชุดทดสอบที่ผ่านการตรวจบน Fixed Revision ไปทดสอบกับ Buggy Revision โดยใช้โค้ดเดียวกัน หากมีกรณีทดสอบที่ผ่าน Fixed แต่ไม่ผ่าน Buggy จะบันทึกว่าตรวจพบข้อบกพร่อง เมื่อไม่มีกรณีเปิดเผยความต่างและการรันครบถ้วนจึงบันทึกว่าไม่พบ หากคอมไพล์ไม่ผ่าน เกิด timeout หรือเครื่องมือไม่สามารถให้ผลที่ตีความได้ จะบันทึกสถานะข้อผิดพลาดและไม่รวมรายการนั้นเป็นข้อบกพร่องที่ประเมินแล้ว

ตารางที่ 5 หลักเกณฑ์จำแนกผลการตรวจชุดทดสอบ

สถานะ	ความหมายในการประเมิน
complete และ fault_detected = true	ชุดทดสอบผ่านเกณฑ์บน Fixed และเปิดเผยความต่างบน Buggy
complete และ fault_detected = false	ประเมินได้ครบแต่ไม่พบความต่างระหว่างรุ่นตามเกณฑ์
no_generated_tests หรือ no_valid_tests	ไม่มีชุดทดสอบที่นำไปประเมินได้
validation_failed, timeout หรือ error	ผลไม่เพียงพอสำหรับตัดสินการตรวจพบบั๊ก ต้องดู status_detail และ logs

รายละเอียดการตรวจของแต่ละวิธีต่างกันในส่วนการสร้าง oracle และการรันซ้ำ DE replay คำตอบบน Fixed และตรวจชุดทดสอบที่ผ่านแล้วบนทั้งสองรุ่นซ้ำเพื่อยืนยันผล ACO ใช้ผล Fixed สองครั้งเป็น oracle แล้วกรอง JUnit ที่ไม่ผ่าน ส่วน Claude และ Gemini สร้าง assertion จากโค้ดและพรีคอมต์ก่อนนำมาตรวจบน Defects4J ดังนั้นผลการเปรียบเทียบต้องอธิบายวิธีสร้าง oracle ร่วมกับค่า coverage และอัตราตรวจพบบั๊ก

3.6 การวัดผลตามตัวชี้วัดที่กำหนด

โครงการใช้ตัวชี้วัดหลักสามรายการ ได้แก่ Line Coverage, Condition Coverage และ Fault Detection Rate การวัดผลใช้รายการข้อบกพร่องของ Defects4J ที่กำหนดไว้ และบันทึกผลแยกตามวิธี โครงการและหมายเลขข้อบกพร่อง

ตารางที่ 6 ตัวชี้วัดหลักที่ใช้เปรียบเทียบ

ตัวชี้วัด	นิยามและวิธีอ่านผล
Line Coverage	สัดส่วนบรรทัดที่ชุดทดสอบเรียกทำงานที่วัดได้ทั้งหมดในขอบเขตเดียวกัน
Condition Coverage	สัดส่วนผลลัพธ์ของเงื่อนไขย่อยที่ครอบคลุมต่อผลลัพธ์ทั้งหมดที่เครื่องมือ Cobertura ระบุ
Fault Detection Rate	สัดส่วนจำนวนบั๊กไม่ซ้ำที่ตรวจพบต่อจำนวนบั๊กไม่ซ้ำที่ประเมินผลได้

เมื่อสรุปผลต้องรายงานจำนวนบั๊กที่เลือก จำนวนที่ประเมินได้ และจำนวนที่ตรวจพบควบคู่กับอัตราการตรวจพบ กรณีไม่มีข้อมูลหรือประเมินไม่ได้ให้คงสถานะดังกล่าวไว้ ไม่แทนค่าด้วยศูนย์ เพราะศูนย์หมายถึงมีการวัดแล้วแต่ไม่พบผล ส่วนผลหลายรอบให้ระบุชัดว่าเป็นค่าเฉลี่ยต่อรอบหรือจำนวนบั๊กที่พบอย่างน้อยหนึ่งรอบ

3.7 รวบรวม ตรวจสอบ และเปรียบเทียบผลการทดลอง

ผลรายบักของแต่ละวิธีถูกจัดเก็บแยกตาม experiment, algorithm, project, bug_id และ run โดยมี result.json เป็นข้อมูลหลัก พร้อมชุดทดสอบ การตั้งค่า และบันทึกคำสั่งที่ใช้ โดยการแยกไฟล์เหล่านี้ทำให้ตรวจสอบย้อนกลับได้ว่าแต่ละค่ามาจากชุดทดสอบใด

ตารางที่ 7 หลักฐานผลการทดลองแยกตามวิธี

วิธี	หลักฐานหลักที่ใช้ตรวจสอบ
DE	Configuration, ผลรายบัก, ชุดทดสอบที่สร้าง, configuration.json, environment.json และ manifest.json
ACO	aco_config.properties, specs.tsv, search_stats.json, AcoGeneratedTest.java, validated_tests และ result.json
Claude	system/user prompt, prompt history, raw_response.txt, generation.json, ชุดทดสอบและ result.json
Gemini	template prompt, ซอร์สใน dataset, ชุดทดสอบใน test/, ผล validator และ result.json

ทำการตรวจสอบผลก่อนนำไปคำนวณ โดยพิจารณาว่าชุดทดสอบคอมไพล์และรันได้ ผลความครอบคลุมมีข้อมูลครบถ้วน และสามารถระบุได้ว่าชุดทดสอบตรวจพบข้อบกพร่องหรือไม่ หากการทดลองรายการใดล้มเหลวหรือไม่มีค่าที่วัดได้ จะบันทึกสาเหตุและแยกออกจากการคำนวณตัวชี้วัดนั้น

จากนั้นสรุปผลของแต่ละวิธีด้วยตัวชี้วัดสามรายการ เปรียบเทียบข้ามวิธีจะใช้ข้อบกพร่องรายการเดียวกัน และขอบเขตการวัดเดียวกัน พร้อมระบุจำนวนรายการที่ใช้คำนวณในแต่ละตาราง เพื่อให้เห็นความแตกต่างของผลลัพธ์อย่างชัดเจนและไม่เปรียบเทียบค่าเฉลี่ยที่ได้

แล้วพิจารณาผลตัวชี้วัดร่วมกับจำนวนชุดทดสอบที่ใช้งานได้และปัญหาที่พบระหว่างการทดลองของแต่ละวิธี เพื่ออธิบายจุดเด่นและข้อจำกัด ผลการเปรียบเทียบนี้จะนำเสนอและอภิปรายในบทที่ 4

บทที่ 4

ผลการทดลองและการวิเคราะห์ผล

ผลการสร้างชุดทดสอบของ Differential Evolution (DE), Ant Colony Optimization (ACO), Claude และ Gemini จากรายการข้อบกพร่องของ Defects4J จำนวน 854 รายการ ผลที่รายงานจำนวนใหม่จากไฟล์ result.json รายการข้อบกพร่องของแต่ละวิธี โดยแยกความครบถ้วนของการทดลองออกจากผลการวัด และใช้รายการเดียวกันเมื่อต้องการเปรียบเทียบประสิทธิภาพระหว่างวิธี

4.1 ข้อมูลที่ใช้วิเคราะห์และเกณฑ์คัดเลือก

ผลการทดลองของแต่ละวิธีมีไฟล์รายการข้อบกพร่อง 854 ไฟล์ และไม่มีหมายเลขข้อบกพร่องซ้ำภายในวิธีเดียวกัน การวิเคราะห์นับหนึ่งข้อบกพร่องต่อหนึ่งวิธีจาก run 1 โดยอ่านสถานะ ผลการตรวจพบข้อบกพร่อง และค่า coverage จากไฟล์รายการข้อบกพร่อง ไม่ใช่ตัวเลขในไฟล์ summary เป็นแหล่งคำนวณหลัก เนื่องจากไฟล์สรุปบางชุดจัดการสถานะที่ไม่ชัดเจนหรือเครื่องมือ coverage ต่างจากนิยามที่ใช้ในบทนี้

รายการที่มีสถานะ complete และมีผล fault_detected เป็น true หรือ false ถือว่าประเมินการตรวจพบข้อบกพร่องได้ ค่า Line Coverage และ Condition Coverage คำนวณเฉลี่ยเฉพาะรายการ complete ที่มีค่าตัวเลขและวัดด้วย Cobertura ในขอบเขต modified classes ค่า null หมายถึงไม่มีผลที่ใช้คำนวณ ไม่ตีความเป็นศูนย์

4.2 ความครบถ้วนของการทดลอง

ตารางที่ 8 สถานะผลรายการข้อบกพร่องของทั้งสี่วิธี

วิธี	complete	สถานะอื่น	รวม
DE	701	no_generated_tests 153	854
ACO	844	no_generated_tests 7; validation_failed 3	854
Claude	697	no_valid_tests 153; validation_failed 3; timeout 1	854
Gemini	455	validation_failed 399	854

ACO มีรายการสถานะ complete มากที่สุด 844 รายการ รองลงมาคือ DE 701 รายการ Claude 697 รายการ และ Gemini 455 รายการ อย่างไรก็ตาม สถานะ complete ของ Gemini ไม่ได้หมายความว่าตัดสินใจการตรวจพบข้อบกพร่องได้ทุกรายการ: 290 จาก 455 รายการมี fault_detected เป็น null โดยส่วนหนึ่งมีผลทดสอบบน Fixed Version ที่ไม่ผ่าน จึงต้องแยกออกจากตัวหารของ Fault Detection Rate

สำหรับ ACO และ Claude ยังพบรายการสถานะ validation_failed ที่มีค่า fault_detected เป็น false อยู่บางรายการ การวิเคราะห์นี้ยึดเงื่อนไข complete เพื่อป้องกันการนับผลที่กระบวนการตรวจสอบไม่สมบูรณ์เป็นการตรวจแล้วไม่พบข้อบกพร่อง

4.3 ผลตัวชี้วัดจากรายการที่แต่ละวิธีประเมินได้

ตารางที่ 9 ผลรายวิธีจากข้อมูลที่ประเมินได้ของแต่ละวิธี

วิธี	Line Coverage	Condition Coverage	ตรวจพบ / ประเมินได้	Fault Detection Rate
DE	44.41% (n=698)	33.06% (n=693)	95 / 701	13.55%
ACO	46.81% (n=844)	35.08% (n=840)	165 / 844	19.55%
Claude	72.61% (n=697)	63.89% (n=692)	191 / 697	27.40%
Gemini	68.87% (n=453)	59.45% (n=449)	10 / 165	6.06%

โดยค่า coverage ในตารางที่ 9 เป็นค่าเฉลี่ยของอัตราส่วนรายข้อบกพร่อง ไม่ใช่อัตราส่วนที่รวมจำนวนบรรทัดหรือเงื่อนไขของทุกโครงการเข้าด้วยกัน จำนวน n จึงแตกต่างกันตามความครบถ้วนของข้อมูล โดย DE มี 3 รายการที่ใช้ JaCoCo และไม่นำมาคำนวณ Condition Coverage ส่วนบางรายการของทุกวิธีไม่มีเงื่อนไขที่วัดได้ จึงมีจำนวนข้อมูล Condition Coverage ต่ำกว่าจำนวนข้อมูล Line Coverage

ตารางนี้แสดงผลที่แต่ละวิธีทำได้ในรายการที่ประเมินได้ของตนเอง ซึ่งแต่ขนาดและองค์ประกอบของกลุ่มข้อบกพร่องแตกต่างกัน โดยเฉพาะ Gemini ซึ่งมีผลตัดสินการตรวจพบข้อบกพร่องได้ 165 รายการจาก 854 รายการ การจัดอันดับประสิทธิภาพจากตารางนี้เพียงตารางเดียวจึงอาจคลาดเคลื่อน

4.4 การเปรียบเทียบบนรายการข้อบกพร่องเดียวกัน

เลือกโครงการ Math ของ Defects4J และใช้รายการ Math-1 ถึง Math-106 ทั้งหมดเป็นขอบเขตเปรียบเทียบเดียวกันสำหรับ DE, ACO, Claude และ Gemini แต่ละวิธีมี result.json ครบ 106 รายการ การมีไฟล์ผลไม่ได้หมายความว่าทุกรายการสร้างชุดทดสอบหรือประเมินการตรวจพบข้อบกพร่องได้ จึงรายงานทั้งจำนวนรายการที่มีสถานะ complete จำนวนที่ตัดสิน fault_detected ได้ และจำนวนที่ไม่มีผลตัดสิน

ตารางที่ 10 ความครบถ้วนของผลในโครงการ Math จำนวน 106 ข้อบกพร่อง

วิธี	complete	ประเมิน fault ได้	ตรวจพบ	ไม่ทราบผล	สถานะอื่น
DE	96	96	17	10	no_generated_tests 10
ACO	103	103	27	3	no_generated_tests 3
Claude	103	103	33	3	no_valid_tests 3
Gemini	76	34	4	72	validation_failed 30

Gemini มีสถานะ complete 76 รายการ แต่ในจำนวนนี้ 42 รายการมี fault_detected เป็น null จึงประเมินการตรวจพบข้อบกพร่องได้ 34 รายการ เมื่อรวมกับ validation_failed อีก 30 รายการ จะมี 72 รายการที่ยังไม่ทราบผล การแสดงจำนวนดังกล่าวป้องกันไม่ให้เกิดความรายการที่ขาดผลว่าเป็นรายการที่ทดสอบแล้วแต่ตรวจไม่พบ

ตารางที่ 11 ตัวชี้วัดของโครงการ Math จากเป้าหมายเดียวกัน 106 รายการ

วิธี	Line Coverage	Condition Coverage	Fault Detection Rate	ตรวจพบต่อเป้าหมาย
DE	53.24% (n=96)	43.00% (n=96)	17/96 = 17.71%	17/106 = 16.04%
ACO	58.80% (n=103)	46.78% (n=103)	27/103 = 26.21%	27/106 = 25.47%
Claude	80.67% (n=103)	72.51% (n=103)	33/103 = 32.04%	33/106 = 31.13%
Gemini	83.18% (n=76)	74.86% (n=76)	4/34 = 11.76%	4/106 = 3.77%

โดยค่า Line Coverage และ Condition Coverage เป็นค่าเฉลี่ยของอัตราส่วนรายข้อบกพร่องที่มีผล Cobertura และสถานะ complete เท่านั้น โดยมีจำนวน n กำกับทุกค่า ส่วน Fault Detection Rate ใช้จำนวนที่ตรวจพบหารด้วยจำนวนที่ตัดสินผลได้ตามนิยามในบทที่ 3 คอลัมน์ “ตรวจพบต่อเป้าหมาย” ใช้ 106 เป็นตัวหารร่วมเพื่อแสดงผลผลิตจากเป้าหมายทั้งหมด จึงมีความหมายต่างจาก Fault Detection Rate และไม่ถือว่ารายการที่ยังไม่ทราบผลเป็นการตรวจไม่พบ

ซึ่งจาก 106 รายการ Claude ตรวจพบมากที่สุด 33 รายการ รองลงมาคือ ACO 27 รายการ DE 17 รายการ และ Gemini 4 รายการ ค่า coverage ของ Gemini สูงที่สุดในรายการที่วัดได้ แต่มีผล coverage 76 รายการและผลตัดสิน fault เพียง 34 รายการ ขณะที่อีกสามวิธีมีผลตัดสิน 96–103 รายการ จึงไม่ควรอ่านค่า coverage เฉลี่ยจากคนละกลุ่มรายการเป็นการจัดอันดับโดยตรง การเปรียบเทียบนี้แสดงทั้งผลที่วัดได้และช่องว่างของข้อมูลบนเป้าหมาย Math ชุดเดียวกัน

4.5 การอภิปรายผลและปัจจัยที่มีผลต่อการตีความ

4.5.1 ความสัมพันธ์ระหว่าง coverage กับการตรวจพบข้อบกพร่อง

โดย Line Coverage และ Condition Coverage บอกว่าส่วนใดของโปรแกรมถูกเรียกใช้ แต่ไม่บอกว่าชุดทดสอบตรวจผลลัพธ์ได้ถูกต้องเพียงใด ในโครงการ Math ค่า coverage เฉลี่ยของ Gemini สูงในรายการที่วัดได้ แต่ตรวจพบข้อบกพร่อง 4 รายการจากเป้าหมาย 106 รายการ และตัดสินผลได้เพียง 34 รายการ จึงต้องพิจารณาทั้งคุณภาพของ assertion และความครบถ้วนของการประเมินประกอบกัน

4.5.2 ความครบถ้วนของชุดทดสอบและผลตรวจสอบ

จำนวนรายการที่สร้างชุดทดสอบได้และจำนวนรายการที่ตรวจพบข้อบกพร่องเป็นคนละประเด็น DE มี 153 รายการที่ไม่สร้างชุดทดสอบ Claude มี 153 รายการที่ไม่มีชุดทดสอบผ่านการตรวจสอบ ส่วน Gemini มี 399 รายการที่ validation_failed และอีก 290 รายการที่มีสถานะ complete แต่ไม่สามารถตัดสิน fault_detected ได้ การรายงานเพียงร้อยละของรายการที่ประเมินได้จึงต้องแสดงจำนวนรายการที่ไม่ทราบผลประกอบเสมอ

4.5.3 ข้อจำกัดของการทดลอง

ผลที่ทำการตรวจสอบเป็น run 1 ของแต่ละวิธี จึงยังไม่แสดงความแปรปรวนจากการสุ่มหรือการเรียกใช้โมเดลหลายครั้ง แม้การเปรียบเทียบโครงการ Math ใช้เป้าหมายเดียวกันครบ 106 รายการ แต่ค่าเฉลี่ย coverage และ Fault Detection Rate มีจำนวนรายการที่คำนวณได้ต่างกัน นอกจากนี้ การทดลองแต่ละวิธีใช้รูปแบบการสร้างชุดทดสอบและการกำหนดผลที่คาดหวังต่างกัน จึงไม่สรุปความแตกต่างเชิงสาเหตุหรือความมีนัยสำคัญทางสถิติ

4.6 ข้อจำกัดของแต่ละวิธี

การพิจารณาข้อจำกัดต่อไปนี้อิงกระบวนการสร้างชุดทดสอบที่ระบุในโครงการและผลของ Math ทั้ง 106 รายการ โดยข้อจำกัดเป็นเงื่อนไขที่พบจากผลการทดลองหรือจากขอบเขตของวิธี ไม่ใช่ค่าความครอบคลุมเพียงค่าเดียวตัดสินคุณภาพ

4.6.1 Differential Evolution

ข้อจำกัดคือการแทนข้อมูลนำเข้าเป็นคำตอบสำหรับการค้นหาไม่ครอบคลุมพฤติกรรมที่ต้องสร้างวัตถุซับซ้อนหรือลำดับสถานะหลายขั้นตอนได้เท่ากันทุกเมธอด ผล Math มี 10 รายการที่ไม่สร้างชุดทดสอบ และค่า coverage เฉลี่ยของรายการที่วัดได้ต่ำกว่า ACO, Claude และ Gemini ในชุดผลนี้ การเปลี่ยนชนิดข้อมูลหรือขอบเขตการค้นหาอาจต้องปรับการเข้ารหัสคำตอบและค่ากำหนดของ DE เพิ่มเติม

4.6.2 Ant Colony Optimization

ข้อจำกัดคือชุดทดสอบที่สร้างจากการเรียกเมธอดหนึ่งครั้งอาจเข้าถึงพฤติกรรมที่ต้องเรียกหลายเมธอดต่อเนื่องบนวัตถุเดียวกันได้ไม่ครบ การสร้างออบเจกต์ผ่าน reflection และการกำหนดเวลาสูงสุดต่อการเรียกยังอาจทำให้บางเส้นทางไม่ถูกสำรวจ ผล Math มี 3 รายการที่ไม่สร้างชุดทดสอบ และค่า coverage เฉลี่ยต่ำกว่าวิธีที่ใช้โมเดลภาษาในรายการที่วัดได้

4.6.3 Claude

ข้อจำกัดคือโค้ดทดสอบที่โมเดลสร้างอาจคอมไพล์ไม่ผ่าน อ้าง API ไม่ตรง หรือสร้าง assertion ที่ไม่ผ่าน Fixed Version จึงต้องมีขั้นตอนคัดกรอง ใน Math มี 3 รายการที่ไม่มีชุดทดสอบผ่านการตรวจสอบ การใช้โมเดลยังขึ้นกับพารามิเตอร์ รุ่นโมเดล และบริการที่เรียกใช้ ทำให้การทำซ้ำต้องบันทึกข้อมูลเหล่านี้พร้อมคำตอบดิบ และมีต้นทุนการเรียกใช้ที่ต้องพิจารณา

4.6.4 Gemini

ข้อจำกัดที่เด่นชัดคือความครบถ้วนของการประเมิน Math มี 30 รายการ validation_failed และใน 76 รายการสถานะ complete ยังมี 42 รายการที่ fault_detected เป็น null จึงตัดสินผลได้เพียง 34 รายการและตรวจพบ 4 รายการ นอกจากนี้ การรันซ้ำต้องตรวจ path ของ template, โพลเดอร์ชุดทดสอบ และตัวเลือกของสคริปต์ให้ตรงกับโครงสร้างปัจจุบัน ตลอดจนบันทึกรุ่นโมเดลและการตั้งค่าบริการเพื่อให้ตรวจสอบผลย้อนหลังได้

บทที่ 5

สรุปผลและข้อเสนอแนะ

5.1 สรุปผลการศึกษา

โครงการนี้ศึกษาการสร้างชุดทดสอบ Java อัตโนมัติด้วยวิธี DE, ACO, Claude และ Gemini บนข้อบกพร่องของ Defects4J จำนวน 854 รายการต่อวิธี จากไฟล์ผลรายข้อบกพร่อง พบว่าทั้งสี่วิธีสร้างหลักฐานการทดลองครบตามจำนวนรายการเป้าหมาย แต่จำนวนที่ประเมินผลได้แตกต่างกัน โดย ACO มีสถานะ complete 844 รายการ DE 701 รายการ Claude 697 รายการ และ Gemini 455 รายการ

เมื่อเปรียบเทียบโครงการ Math ทั้ง 106 ข้อบกพร่อง Claude ตรวจพบ 33 รายการจาก 103 รายการที่ตัดสินผลได้ (32.04%) ACO ตรวจพบ 27 จาก 103 (26.21%) DE ตรวจพบ 17 จาก 96 (17.71%) และ Gemini ตรวจพบ 4 จาก 34 (11.76%) เมื่อนับจำนวนตรวจพบเทียบกับเป้าหมาย 106 รายการ Claude ได้ 31.13% ACO 25.47% DE 16.04% และ Gemini 3.77% โดยรายการที่ไม่มีผลตัดสินยังคงรายงานแยกไว้

ผลทดลองยังแสดงว่าความครอบคลุมของโค้ดและการตรวจพบข้อบกพร่องไม่เพิ่มขึ้นไปในทิศทางเดียวกันเสมอ Gemini มีค่า coverage เฉลี่ยสูงในรายการที่วัดได้ แต่มีผลตัดสินการตรวจพบเพียง 34 จาก 106 รายการ การประเมินคุณภาพชุดทดสอบจึงต้องพิจารณาทั้งการเข้าถึงโค้ด ความถูกต้องของผลที่คาดหวัง และความครบถ้วนของข้อมูล

เมื่อพิจารณาร่วมกับวิธีทำงาน DE และ ACO มีค่ากำหนดการค้นหาที่ตรวจสอบได้ โดย ACO ให้ผลประเมินครบกว่าใน Math ส่วน Claude ตรวจพบข้อบกพร่องมากที่สุดแต่ต้องคัดกรองชุดทดสอบที่สร้างจากโมเดล Gemini ให้ค่า coverage สูงในรายการที่วัดได้แต่ยังมีผลตัดสินการตรวจพบไม่ครบจำนวนมาก ข้อจำกัดของแต่ละวิธีจึงสัมพันธ์กับทั้งวิธีสร้างข้อมูลนำเข้า คุณภาพของ assertion และความพร้อมของกระบวนการตรวจสอบ

5.2 ข้อจำกัดของงาน

ข้อจำกัดสำคัญคือจำนวนผลที่ตัดสินได้ต่างกันมากระหว่างในแต่ละวิธี โดยเฉพาะที่ Gemini ซึ่งตรวจจับ fault_detected ที่เป็น true หรือ false เพียง 165 รายการ แม้มีสถานะ complete 455 รายการ การรายงาน Fault Detection Rate ของ Gemini จึงใช้ 165 รายการเป็นตัวหาร และไม่ถือว่ารายการที่เป็น null เป็นการตรวจไม่พบข้อบกพร่อง

การเปรียบเทียบหลักครอบคลุม Math ทั้ง 106 รายการ แต่จำนวนที่คำนวณ coverage และ Fault Detection Rate ได้แตกต่างกัน โดยเฉพาะ Gemini ซึ่งตัดสินผลได้ 34 รายการ ค่าเฉลี่ยจากรายการที่มีผลจึงอาจไม่สะท้อนข้อบกพร่องทั้งหมด และผลของ Math ไม่อาจแทนผลของโครงการอื่น นอกจากนี้ มีผลเพียงหนึ่งรอบต่อข้อบกพร่อง จึงไม่สามารถประเมินความคงที่ของวิธีที่มีการสุ่มหรือของเครื่องมือ AI ได้

เครื่องมือวัด coverage และวิธีสร้าง oracle แตกต่างกันบางส่วน DE มีการใช้ JaCoCo เมื่อ Cobertura ใช้งานไม่ได้ ส่วนวิธีที่ใช้ AI สร้าง assertion จากพรมบ่งมีความเสี่ยงที่ชุดทดสอบจะคอมไพล์ไม่ผ่านหรือคาดหวังผลไม่ตรงกับ Fixed Version ความแตกต่างเหล่านี้จำกัดการอธิบายว่าตัววิธีสร้างชุดทดสอบเป็นสาเหตุโดยตรงของผลที่ต่างกัน

5.3 ข้อเสนอแนะสำหรับการพัฒนาต่อ

ควรปรับกระบวนการตรวจสอบชุดทดสอบให้สามารถแยกสถานะ complete ออกจากสถานะที่ติดสิน fault_detected ไม่ได้อย่างชัดเจน โดยเฉพาะรายการที่ Fixed Version ยังล้มเหลว เพื่อให้ตัวหารของ Fault Detection Rate ที่ตรวจสอบได้จากสถานะโดยตรง และควรสร้างไฟล์สรุปรูปใหม่จาก result.json หลังการปรับรูปแบบข้อมูลทุกครั้ง

ควรทดลองซ้ำหลายรอบด้วย seed หรือคำตอบจากโมเดลที่แตกต่างกัน แล้วรายงานทั้งค่าเฉลี่ยและการกระจายของ Line Coverage, Condition Coverage และ Fault Detection Rate การออกแบบรอบทดลองควรกำหนดเวลา งบประมาณการค้นหา และจำนวนการเรียกใช้โมเดลให้ชัดเจน เพื่อช่วยการตีความความต่างด้านประสิทธิภาพและต้นทุนของแต่ละวิธี

ควรขยายการเปรียบเทียบไปยังข้อบกพร่องร่วมที่มีผลสมบูรณ์มากขึ้น โดยปรับปรุงปัญหาการสร้างชุดทดสอบและการตรวจสอบบน Fixed Version ของแต่ละวิธีการทดสอบ พร้อมกำหนดเครื่องมือ coverage และขอบเขต modified classes ให้สอดคล้องกัน

ปรับปรุงวิธีสร้างชุดทดสอบ ควรเพิ่มความสามารถสร้างลำดับการเรียกหลายเมธอดบนวัตถุเดียวกัน เพื่อรองรับพฤติกรรมที่ขึ้นกับสถานะ และปรับปรุงการสร้าง oracle ให้ตรวจผลลัพธ์สำคัญได้มากขึ้น การวิเคราะห์รายการที่วิธีหนึ่งตรวจพบแต่อีกวิธีไม่พบ เพื่อให้อธิบายในการระบุความต่างเกิดจากข้อมูลนำเข้า เส้นทางที่เข้าถึง หรือ assertion ของชุดทดสอบ

5.4 ข้อสรุป

จากหลักฐานที่ตรวจสอบได้ การใช้ตัวชี้วัดทั้งสามร่วมกับจำนวนรายการที่ประเมินได้ช่วยให้เห็นทั้งความครอบคลุมและความสามารถในการเปิดเผยข้อบกพร่องของชุดทดสอบ ในโครงการ Math ทั้ง 106 รายการ Claude ตรวจพบข้อบกพร่องมากที่สุด ส่วน Gemini มีค่า coverage เฉลี่ยสูงในกลุ่มที่วัดได้แต่มีผลที่ประเมินได้จำกัด ขณะที่ ACO มีจำนวนรายการสถานะ complete มากที่สุดเมื่อพิจารณาข้อมูลทั้ง 854 รายการ การนำผลไปใช้เลือกวิธีสำหรับงานอื่นควรพิจารณาลักษณะโครงการ คุณภาพของ oracle ความพร้อมของเครื่องมือ และข้อจำกัดของข้อมูลที่ใช้เปรียบเทียบประกอบกัน